

Computational Complexity (CPS-MIRI)  
CP assignment  
Report

Roger Canal Estrada, [roger.canal@estudiantat.upc.edu](mailto:roger.canal@estudiantat.upc.edu)  
[CPS git repository](#)

Course 2025/2026

# Introduction

This is the report for the Integer Linear Programming (ILP) model to solve the Street Directionality Problem (SDP). I will jump directly to the variables, then the constraints, the objective function, and some implementation notes. The code is in my public [GitHub repository](#).

## Variables

### Decision Variables

- **streets\_state** (*IloBoolVarArray* of size  $n \times n$ ):  
This binary decision array represents the operational directed graph layout. Each element maps to coordinates via the linear layout  $i \cdot n + j$ .

$$\text{streets\_state}_{i \cdot n + j} = \begin{cases} 1 & \text{if the directed lane } i \rightarrow j \text{ is open to traffic} \\ 0 & \text{if the lane is closed (parking)} \end{cases} \quad (1)$$

### Auxiliary Variables

- **current\_paths** (*4 dimensions vector of IloNumVar*):  
This variable is a bit tricky. The idea is to have track of the paths for every source-destination pair. For every active source-destination pair  $(s, d)$  across every physical street segment  $(i, j)$ :

$$\text{current\_paths}[s][d][i][j] = \begin{cases} 1 & \text{if the unique path from } (s, d) \text{ uses lane } i \rightarrow j \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

That is why it uses four dimensions: the first two for the source and destination, and the last two for the physical street segments.

### Objective Variable

- **converted\_expr** (*IloExpr*):  
A linear expression tracking the total quantity of converted two-way streets. Its values range between 0 and the result of the preprocessing of the input (**total\_two\_way**).

## Constraints

### Street Constraints

For every unique node pair  $(i, j)$  where  $j > i$ :

- **Original Two-Way Streets:** At least one directional lane must remain open to traffic to avoid cutting off connections entirely:

$$\text{streets\_state}_{i \cdot n + j} + \text{streets\_state}_{j \cdot n + i} \geq 1 \quad (3)$$

---

```

if (streets_graph[i][j] != -1 && streets_graph[j][i] != -1) {
    model.add(streets_state[i * n + j] + streets_state[j * n + i] >= 1);
}

```

---

- **Original One-Way Streets** ( $i \rightarrow j$ ): Cannot be modified. They are clamped permanently to their initial direction:

$$\text{streets\_state}_{i \cdot n + j} = 1, \quad \text{streets\_state}_{j \cdot n + i} = 0 \quad (4)$$

---

```

if (streets_graph[i][j] != -1 && streets_graph[j][i] == -1) {
    model.add(streets_state[i * n + j] == 1);
    model.add(streets_state[j * n + i] == 0);
} else if (streets_graph[i][j] == -1 && streets_graph[j][i] != -1) {
    model.add(streets_state[i * n + j] == 0);
    model.add(streets_state[j * n + i] == 1);
}

```

---

- **Non-existent Connections:** Forced to zero across both directional mappings:

$$\text{streets\_state}_{i \cdot n + j} = 0, \quad \text{streets\_state}_{j \cdot n + i} = 0 \quad (5)$$

---

```

if (streets_graph[i][j] == -1 || streets_graph[j][i] == -1) {
    model.add(streets_state[i * n + j] == 0);
    model.add(streets_state[j * n + i] == 0);
}

```

---

## Path Validity

A route cannot be transmit if the street is not active. For every source-destination pair  $(s, d)$  and every street segment  $(i, j)$ :

$$\text{current\_paths}[s][d][i][j] \leq \text{streets\_state}_{i \cdot n + j} \quad \forall (s, d), \forall (i, j) \quad (6)$$

## Continuous Path Constraints

To ensure an uninterrupted path from each source  $s$  to destination  $d$ , every node  $i$  evaluates its balancing net flow (outgoing flow - incoming flow): Let define the net flow at node  $i$  for the source-destination pair  $(s, d)$  as:

$$\text{outflow}_i = \sum_{j: T_{i,j} \neq -1} \text{current\_paths}[s][d][i][j] \quad (7)$$

$$\text{inflow}_i = \sum_{j: T_{j,i} \neq -1} \text{current\_paths}[s][d][j][i] \quad (8)$$

$$\text{outflow}_i - \text{inflow}_i = \begin{cases} 1 & \text{if } i = s \quad (\text{Source Node}) \\ -1 & \text{if } i = d \quad (\text{Destination Node}) \\ 0 & \text{otherwise} \quad (\text{Intermediate Crossings}) \end{cases} \quad (9)$$

## Time Limit Constraints

Using the precalculated matrices from the input, the cumulative edge weights of active path indicators must stay within bounded limits:

$$\sum_{i=0}^{n-1} \sum_{j=0}^{n-1} (\text{streets\_graph}[i][j] \times \text{current\_paths}[s][d][i][j]) \leq \text{max\_time\_allowed\_matrix}[s][d] \quad (10)$$

## Objective Calculation

The objective function is to maximize the number of conversions. A way to calculate this is to sum the contributions from each street segment, and to calculate the contribution of each segment can be done as follows:

$$\text{Maximize } \sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} (2 - (\text{streets\_state}_{i \cdot n + j} + \text{streets\_state}_{j \cdot n + i})) \quad (11)$$

If both directions of a two-way street stay open, the contribution is  $2 - (1 + 1) = 0$ . If one lane is closed, the contribution is  $2 - (1 + 0) = 1$ . This way, the objective function will be maximized when we convert as many two-way streets as possible, while still satisfying all the constraints.